1 Метод Гаусса решения СЛАУ

Задача: решить СЛАУ $Ax = b$ методом Гаусса с выбором главного элемента в столбце, где

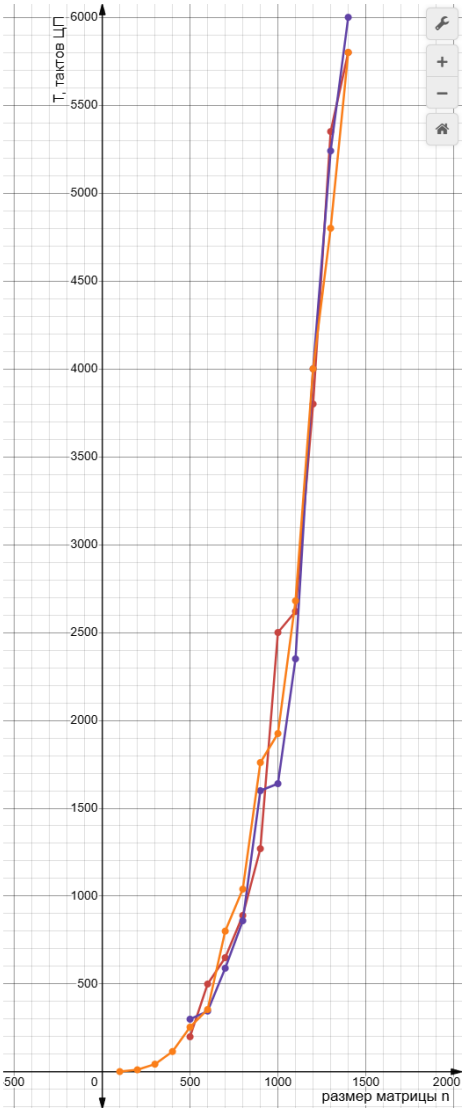
$$A = \begin{pmatrix} 3 & 1 & -1 & 0 & 3 & 2 \\ 1 & -2 & 0.5 & 4 & -1 & 3 \\ 0 & 3 & 1 & -2 & 2 & 1 \\ 1 & 0 & -3 & 2 & 1 & -1 \\ 2 & -1 & 1 & 3 & -2 & 0.5 \\ -1 & 2 & 0 & 1 & 4 & -3 \end{pmatrix} \quad b = \begin{pmatrix} 4 \\ 2 \\ 5 \\ 3 \\ 1 \\ 6 \end{pmatrix}.$$

С помощью написанной программы получено решение, выполнена проверка корректности решения $Ax - b = 0$, построен график зависимости времени выполнения от размера матрицы A.

$$x = \begin{pmatrix} -0.162611 & 2.253364 & -0.166619 & 1.402806 & 0.356427 & 0.499284 \end{pmatrix}^T,$$

```
> .\Gauss_main_el.exe
Введите 0 для генерации матрицы, 1 для чтения из файла: 1

=================
Матрица A|b:
3.00 1.00 -1.00 0.00 3.00 2.00 4.00
1.00 -2.00 0.50 4.00 -1.00 3.00 2.00
0.00 3.00 1.00 -2.00 2.00 1.00 5.00
1.00 0.00 -3.00 2.00 1.00 -1.00 3.00
2.00 -1.00 1.00 3.00 -2.00 0.50 1.00
-1.00 2.00 0.00 1.00 4.00 -3.00 6.00
Матрица после прямого хода (верхнетреугольная):
3.000 1.000 -1.000 0.000 3.000 2.000 4.000
0.000 3.000 1.000 -2.000 2.000 1.000 5.000
0.000 0.000 -2.556 1.778 0.222 -1.556 2.222
0.000 0.000 0.000 3.565 -0.304 2.130 5.957
0.000 0.000 0.000 0.000 3.500 -3.500 -0.500
0.000 0.000 0.000 0.000 0.000 -6.085 -3.038
Время выполнения: 0.001000 секунд
Процессорные такты: 1
Решение системы:
X[0] = -0.162611
X[1] = 2.253364
X[2] = -0.166619
X[3] = 1.402806
X[4] = 0.356427
X[5] = 0.499284
=================
check[0] = 0.000000
check[1] = 0.000000
check[2] = -0.000000
check[3] = 0.000000
check[4] = -0.000000
check[5] = -0.000000
```



Программная реализация на языке C:

```c
#include <stdio.h>
#include <locale.h>
#include <stdlib.h>
#include <windows.h>
#include <math.h>
#include <time.h>


double* gauss(double **A, int n) {
    // Прямой ход с выбором главного элемента по столбцу
    for (int k = 0; k < n - 1; k++) {
        // 1. Поиск главного элемента в столбце k
        int max_row = k;
        double max_val = fabs(A[k][k]);
        for (int i = k + 1; i < n; i++) {
            if (fabs(A[i][k]) > max_val) {
                max_val = fabs(A[i][k]);
                max_row = i;
            }
        }
        // Проверка на ноль
        if (max_val < 1e-12) {
            printf("Ошибка: столбец %d почти нулевой (главный элемент = %g)\n",
                 k, max_val);
            return NULL;
        }
        // 2. Перестановка строк
        if (max_row != k) {
            double *tmp = A[k];
            A[k] = A[max_row];
            A[max_row] = tmp;
        }
        // 3. Исключение под диагональю
        for (int i = k + 1; i < n; i++) {
            double factor = A[i][k] / A[k][k];
            for (int j = k; j < n + 1; j++) {
                A[i][j] -= factor * A[k][j];
            }
        }
    }

    // Вывод матрицы после прямого хода
    if (n < 10) {
        printf("Матрица после прямого хода (верхнетреугольная):\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n + 1; j++) {
                printf("%.3f ", A[i][j]);
            }
            printf("\n");
```

```
        }
    }

    // Обратный ход с проверкой диагонали
    double *X = (double *)malloc(n * sizeof(double));
    for (int i = n - 1; i >= 0; i--) {
        if (fabs(A[i][i]) < 1e-12) {
            printf("Ошибка: нулевой диагональный элемент на шаге %d\n", i);
            free(X);
            return NULL;
        }
        X[i] = A[i][n];
        for (int j = i + 1; j < n; j++) {
            X[i] -= A[i][j] * X[j];
        }
        X[i] /= A[i][i];
    }
    return X;
}

double* matrix_x_vector_column(double **A, double *X, int n) {
    double *B = (double *)malloc(n * sizeof(double));
    for (int i = 0; i < n; i++) {
        B[i] = 0.0;
        for (int j = 0; j < n; j++) {
            B[i] += A[i][j] * X[j];
        }
    }
    return B;
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_NUMERIC, "C");

    double **A;
    double **A_for_check;
    double *B;
    double *X;
    double *X_target;
    int n;

    int is_use_file = 0; // 0 - не использовать файл, 1 - использовать файл
    printf("Введите 0 для генерации матрицы, 1 для чтения из файла: ");
    scanf("%d", &is_use_file);

    if (is_use_file == 1) { // ИСПОЛЬЗУЕТСЯ ФАЙЛ
        FILE *fp = fopen("matrixA.txt", "r");
        // Чтение размера матрицы
        if (fscanf(fp, "%d", &n) != 1) {
```

```
            printf("Ошибка чтения размера матрицы\n");
            fclose(fp);
            return 1;
        }
        // A --- Расширенная матрица коэффициентов (n x (n+1))
        A = (double **)malloc(n * sizeof(double *));
        for (int i = 0; i < n; i++) {
            A[i] = (double *)malloc((n + 1) * sizeof(double));
        }
        // Чтение расширенной матрицы A|b из файла
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n + 1; j++) {
                if (fscanf(fp, "%lf", &A[i][j]) != 1) {
                    printf("Ошибка чтения элемента матрицы A[%d][%d]\n",
                        i, j);
                    fclose(fp);
                    return 1;
                }
            }
        }
        //копируем матрицу A для проверки
        A_for_check = (double **)malloc(n * sizeof(double *));
        for (int i = 0; i < n; i++) {
            A_for_check[i] = (double *)malloc((n + 1) * sizeof(double));
            for (int j = 0; j < n + 1; j++) {
                A_for_check[i][j] = A[i][j];
            }
        }
        // Закрытие файла
        fclose(fp);

        // Решение
        printf("\n================\nМатрица A|b:\n");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n + 1; j++) {
                printf("%.2f ", A[i][j]);
            }
            printf("\n");
        }

        // Вызов функции Гаусса
        clock_t start = clock();
        X = gauss(A, n);
        clock_t end = clock();
        double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
        printf("Время выполнения: %.6f секунд\n", time_spent);
        printf("Процессорные такты: %ld\n", (long)(end - start));

        // Вывод решения
        printf("Решение системы:\n");
```

```
        for (int i = 0; i < n; i++) {
            printf("X[%d] = %.6f\n", i, X[i]);
        }
        printf("=================\n");

        double *check = (double *)malloc(n * sizeof(double));
        check = matrix_x_vector_column(A_for_check, X, n);
        for (int i = 0; i < n; i++) {
            check[i] -= A_for_check[i][n]; // вычитаем b[i]
        }
        for (int i = 0; i < n; i++) {
            printf("check[%d] = %.6f\n", i, check[i]);
        }

        // Освобождение памяти
        for (int i = 0; i < n; i++) {
            free(A[i]);
            free(A_for_check[i]);
        }
        free(A);
        free(X);
    }
    else { // НЕ ИСПОЛЬЗУЕТСЯ ФАЙЛ
        int nn = 100;
        //ввод желаемого размера матрицы через консоль
        printf("Введите размер матрицы n: ");
        scanf("%d", &nn);

        for (n = nn; n < 100; n += 100)
        {
            // A --- Расширенная матрица коэффициентов (n x (n+1))
            A = (double **)malloc(n * sizeof(double *));
            for (int i = 0; i < n; i++) {
                A[i] = (double *)malloc((n + 1) * sizeof(double));
            }
            //заполнение матрицы A и B и их объединение в РМК
            X_target = (double *)malloc(n * sizeof(double));

            int a = 1;
            for (int i = 0; i < n; i++) {
                X_target[i] = a;

                for (int j = 0; j < n; j++) {
                    if (i == j)
                        A[i][j] = a;
                    else
                        A[i][j] = 0.1 * a;
                }
                a +=1;
            }
```

```
            B = matrix_x_vector_column(A, X_target, n);

            for (int i = 0; i < n; i++) {
                A[i][n] = B[i];
            }

            if (n < 10) {
                // Решение
                printf("\n=================\nМатрица A|b:\n");
                for (int i = 0; i < n; i++) {
                    for (int j = 0; j < n + 1; j++) {
                        printf("%.2f ", A[i][j]);
                    }
                    printf("\n");
                }
            }

            // Вызов функции Гаусса
            clock_t start = clock();
            X = gauss(A, n);
            clock_t end = clock();
            double time_spent = (double)(end - start) / CLOCKS_PER_SEC;
            printf("Время выполнения: %.6f секунд\n", time_spent);
            printf("Процессорные такты: %ld\n", (long)(end - start));


            // Вывод решения
            printf("Решение системы:\n");
            if (n < 10)
                for (int i = 0; i < n; i++)
                    printf("X[%d] = %.6f\n", i, X[i]);
            //else for (int i = 0; i < n; i++) printf("%.2f ", X[i]);

            printf("=================\n");

            // Освобождение памяти
            for (int i = 0; i < n; i++) {
                free(A[i]);
            }
            free(A);
            free(X);
            free(B);
            free(X_target);
        }
    }

    return 0;
}
```

2 Метод Ньютона решения системы нелинейных уравнений

Задача:

$$\begin{cases} f_1(x,y) = x^2 + y^2 - 4 = 0 \\ f_2(x,y) = x \cdot y - 1 = 0 \end{cases}$$

Решение:

Построив графики уравнений $f_1(x,y) = 0$ и $f_2(x,y) = 0$, можно увидеть, что они пересекаются в четырех точках, которые являются решениями данной системы нелинейных уравнений. Эти точки можно найти численно с помощью метода Ньютона, используя начальное приближение, выбранное вблизи предполагаемых пересечений.

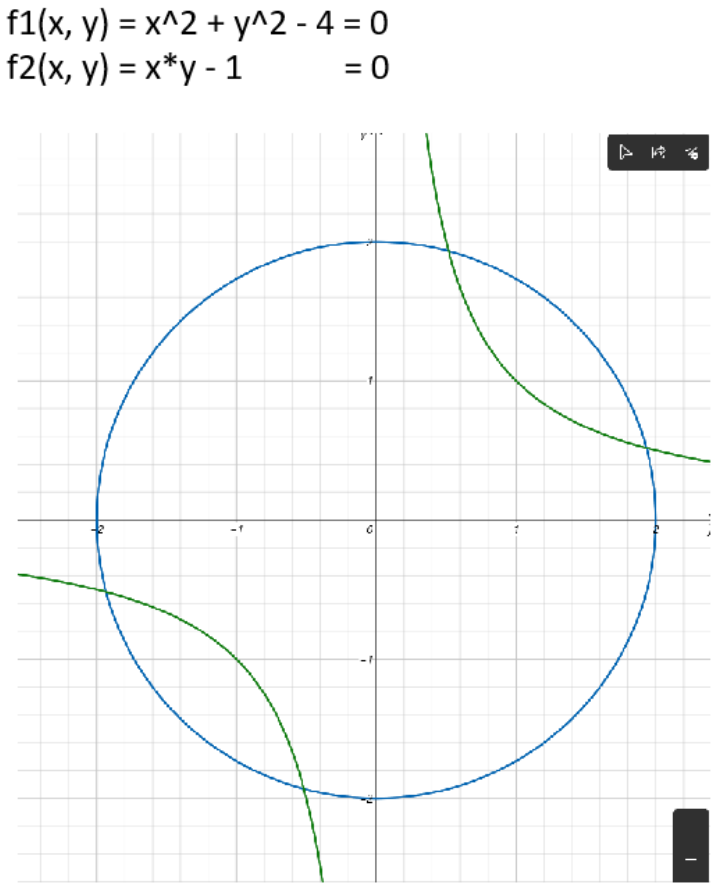


Рисунок 1 – Графики функций $f_1(x,y) = 0$ и $f_2(x,y) = 0$

В результате получаются следующие результаты:

```
> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): 3 2

Итерация 1: x = 2.30000000, y = 0.80000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = 1.96720430, y = 0.55053763, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = 1.93231527, y = 0.51809958, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = 1.93185174, y = 0.51763818, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = 1.9318517399
y = 0.5176381775

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07
```

Рисунок 2 – Начальное приближение (3, 2)

```
> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): 2 3

Итерация 1: x = 0.80000000, y = 2.30000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = 0.55053763, y = 1.96720430, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = 0.51809958, y = 1.93231527, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = 0.51763818, y = 1.93185174, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = 0.5176381775
y = 1.9318517399

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07
```

Рисунок 3 – Начальное приближение (2, 3)

```
> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): -2 -3

Итерация 1: x = -0.80000000, y = -2.30000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = -0.55053763, y = -1.96720430, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = -0.51809958, y = -1.93231527, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = -0.51763818, y = -1.93185174, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = -0.5176381775
y = -1.9318517399

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07
```

Рисунок 4 – Начальное приближение (-2, -3)

```
> .\Newton.exe
Решение системы нелинейных уравнений методом Ньютона:
  f1(x,y) = x^2 + y^2 - 4 = 0
  f2(x,y) = x*y - 1 = 0

Введите начальное приближение (x0 y0): -3 -2

Итерация 1: x = -2.30000000, y = -0.80000000, |Δ| = 1.389244e+00, |F| = 2.104875e+00
Итерация 2: x = -1.96720430, y = -0.55053763, |Δ| = 4.159140e-01, |F| = 1.918748e-01
Итерация 3: x = -1.93231527, y = -0.51809958, |Δ| = 4.763897e-02, |F| = 2.536005e-03
Итерация 4: x = -1.93185174, y = -0.51763818, |Δ| = 6.540277e-04, |F| = 4.782406e-07

Приближённое решение:
x = -1.9318517399
y = -0.5176381775

Количество итераций: 4
Норма разности между двумя последними приближениями: 6.540277e-04
Погрешность (норма невязки |F|): 4.782406e-07
```

Рисунок 5 – Начальное приближение (-3, -2)

Программная реализация на языке C:

```c
#include <stdio.h>
#include <math.h>
#include <locale.h>
#include <stdlib.h>
#include <windows.h>

/* Определение системы нелинейных уравнений:
   f1(x, y) = x^2 + y^2 - 4 = 0
   f2(x, y) = x*y - 1 = 0
*/

double f1(double x, double y) {
    return x * x + y * y - 4.0;
}

double f2(double x, double y) {
    return x * y - 1.0;
}

double df1dx(double x, double y) {
    return 2.0 * x;
}

double df1dy(double x, double y) {
    return 2.0 * y;
}

double df2dx(double x, double y) {
    return y;
}

double df2dy(double x, double y) {
```

```
    return x;
}

int main() {
    SetConsoleOutputCP(CP_UTF8);
    setlocale(LC_NUMERIC, "C");

    double x0, y0, eps;
    int max_iter;

    printf("Решение системы нелинейных уравнений методом Ньютона:\n");
    printf("  f1(x,y) = x^2 + y^2 - 4 = 0\n");
    printf("  f2(x,y) = x*y - 1 = 0\n\n");

    eps = 0.0001; // Точность по умолчанию
    max_iter = 100; // Максимальное число итераций по умолчанию

    // Ввод начального приближения
    printf("Введите начальное приближение (x0 y0): ");
    if (scanf("%lf %lf", &x0, &y0) != 2) {
        printf("Ошибка: некорректный ввод начального приближения.\n");
        return 1;
    }
/*
    // Ввод точности
    printf("Введите точность (eps > 0): ");
    if (scanf("%lf", &eps) != 1 || eps <= 0.0) {
        printf("Ошибка: точность должна быть положительным числом.\n");
        return 1;
    }

    // Ввод максимального числа итераций
    printf("Введите максимальное число итераций: ");
    if (scanf("%d", &max_iter) != 1 || max_iter <= 0) {
        printf("Ошибка: max_iter должно быть целым больше нуля.\n");
        return 1;
    }
*/
    printf("\n");

    double x = x0, y = y0;
    int iter;
    // норма разности между двумя последними приближениями
    double norm_delta_last = 0.0;
    // норма невязки в последней точке
    double norm_F_last = 0.0;

    for (iter = 0; iter < max_iter; ++iter) {
        // Вычисление значений функций и матрицы Якоби
        double f1_val = f1(x, y);
```

```
        double f2_val = f2(x, y);
        double J11 = df1dx(x, y);
        double J12 = df1dy(x, y);
        double J21 = df2dx(x, y);
        double J22 = df2dy(x, y);

        double det = J11 * J22 - J12 * J21;
        if (fabs(det) < 1e-12) {
            printf("Ошибка: якобиан вырожден (det = %e) на итерации %d\n",
                det, iter);
            break;
        }

        // Решение системы J * delta = -F
        double dx = (-J22 * f1_val + J12 * f2_val) / det;
        double dy = (J21 * f1_val - J11 * f2_val) / det;

        double x_new = x + dx;
        double y_new = y + dy;

        // Норма разности между приближениями
        norm_delta_last = sqrt(dx * dx + dy * dy);

        // Вычисление невязки в новом приближении
        double f1_new = f1(x_new, y_new);
        double f2_new = f2(x_new, y_new);
        norm_F_last = sqrt(f1_new * f1_new + f2_new * f2_new);

        // Печать информации об итерации
        printf("Итерация%2d: x = %.8lf, y = %.8lf, |delta| = %e, |F| = %e\n",
               iter + 1, x_new, y_new, norm_delta_last, norm_F_last);

        // Обновление переменных
        x = x_new;
        y = y_new;

        // Проверка критериев остановки
        if (norm_delta_last < eps || norm_F_last < eps) {
            break;
        }
    }

    // Вывод результатов
    printf("\nПриближённое решение:\n");
    printf("x = %.10lf\n", x);
    printf("y = %.10lf\n", y);
    printf("\nКоличество итераций: %d\n", iter + 1);
    printf("Норма разности между двумя последними приближениями: %e\n",
         norm_delta_last);
    printf("Погрешность (норма невязки |F|): %e\n", norm_F_last);
```

```

    if (iter == max_iter && norm_delta_last >= eps && norm_F_last >= eps) {
        printf("\nПредупреждение: достигнуто максимальное число итераций.\n");
    }

    return 0;
}
```

3 Нахождение собственных значений собственых векторов матрицы

Задача: найти собственные значения и собственные векторы матрицы

Программная реализация на языке C:

```


```